

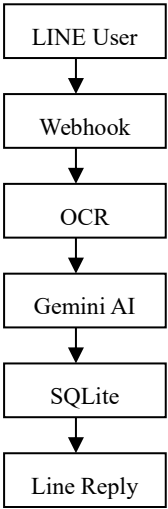
RSA 發票登錄小幫手

1. 專案介紹：

利用 AI 技術，將日常「數據」整理變得簡化方便。

本專案利用到 AI model API，python，HTML，REST API，SQLite，雲端部署等技術。本專案將分為四大關鍵階段：視覺捕捉、智慧解析、結構儲存與介面呈現。

架構流程圖



2. 技術工具箱：構建 AI 助手的五大支柱

要完成這場精密的轉化邏輯，我們需要一套分工明確的技術棧 (Tech Stack)。以下是「Smart Receipt Assistant」系統的核心架構成員：

技術名稱	扮演角色	對學習者的直白比喻
Python	核心開發語言	大腦指揮官 ：負責全域資源調度與邏輯串接。
Flask	Web 網頁框架	溝通橋樑 ：串聯前端介面與後端邏輯的數據轉發站。
Gemini API	AI 視覺與理解模型	LLM-based semantic extraction engine ：不僅識字，更能理解文字背後的含意。
SQLite	輕量化資料庫	persistent storage layer ：負責數據的長期存取與狀態持久化 (State Persistence)。
HTML	網頁標記語言	人機介面 ：提供視覺化的互動視窗。

準備好工具後，我們將從第一階段「視覺捕捉」開始。

3. 第一階段：視覺捕捉與 OCR 識字 (The Vision)

數據轉化的起點是「看見」。原始的 invoice.jpg 包含了複雜的視覺資訊：字體大小不一、兩組 QR Code、黑白的條碼區塊，以及各種背景雜訊。對於人類而言，一眼就能看清內容；但對電腦來說，這最初只是一堆像素點。



uploads/super_invoice.jpg OCR Result:

花蓮市農會自強生鮮超級市場
電子發票證明聯
114年01-02月
WH-37166026
2026-02-12 18:16:35
隨機碼:1885 總計:190
買方:78847073
[Barcode]
[QR Code] [QR Code]
花蓮市農會自強生鮮超級市場
機號:0102 單號:00232 收銀:032
總計:170 (增加:2)

透過 OCR (光學字元辨識) 技術，系統將影像掃描並初步提取為機器可讀的文字流。右圖是 OCR 處理後的原始輸出：

核心洞察：觀察 OCR 結果，你會發現它雖然成功「識字」，卻缺乏「理解能力」。它完整抓取了無意義的 [Barcode] 標籤，且在面對「總計:190」與下方「總計:170」兩個數據時，OCR 無法判斷哪一個才是最終的實付金額。此時的數據處於「高熵」的混亂狀態，這正是我們需要下一步 AI 介入的原因。

此時的文字像是一堆散落的零件，需要一個具備邏輯解析能力的「大腦」來重新組裝。

4. 第二階段：AI 智慧解析與 JSON 定義 (The Brain)

這是轉化流程中最具「智慧」的環節。我們將混亂的 OCR 文字交給 Gemini API 進行「語義建模」。AI 的價值在於它能進行 **Schema Mapping (模式映射)**，從雜訊中過濾出關鍵欄位。

請注意一個關鍵：在 repo003.JPG 中，OCR 同時抓到了 190 與 170 兩個數值。然而，在 repo005.JPG 的 JSON 結果中，AI 展現了它的理解力——它成功識別出 170 才是最終正確的交易總額：

```
{
  "store_name": "花蓮市農會自強生鮮超級市場",
  "invoice_number": "WH-37166026",
  "date": "2026-02-12",
  "total_amount": "170"
}
```

核心洞察：將數據轉化為 JSON (JavaScript Object Notation) 是現代軟體架構的標配，它具備三大優勢：

- **易讀性：**清晰的「鍵值對」(Key-Value pairs) 讓數據意義一目了然。
- **結構化：**將破碎文字歸納入標準化的容器中，方便程式後續運算。
- **標準化：**作為跨系統交換數據的通用語言，JSON 讓資料能無縫傳遞。

雖然 JSON 讓數據變得整齊，但它屬於「易失性記憶」，一旦程式關閉就會消失。因此，我們需要為數據尋找一個「永久的家」-- 資料庫。

5. 第三階段：結構化儲存與 SQL 記憶 (The Memory)

為了達成 **State Persistence (狀態持久化)**，系統會將 JSON 格式的暫存數據寫入 SQLite 資料庫。這是一個從「非結構化」走向「強結構化」的過程。

每一筆發票都會被分配一個自動遞增的 **ID (Primary Key)**。這個 ID 是資料庫邏輯的核心，確保每筆記錄的唯一性。我們可以模擬一條 SQL INSERT 陳述句來理解這個動作：

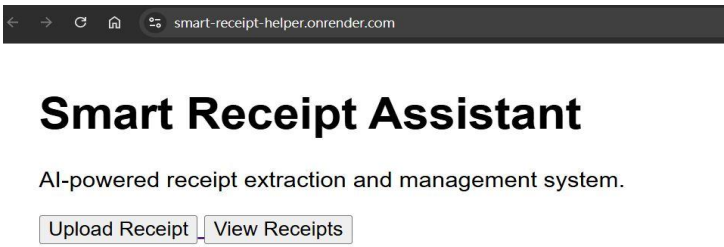
```
-- 將解析後的發票數據持久化存儲，ID 由系統自動生成
INSERT INTO receipts (id, store, invoice, date, amount)
VALUES (2, '花蓮市農會自強生鮮超級市場', 'WH-37166026', '2026-02-12',
170);
```

透過這種方式，原本短暫的 AI 解析結果被轉化為可隨時檢索、統計與維護的長期記憶。

數據已經安全入庫，最後一步是讓使用者能直觀地管理這些成果。

6. 第四階段：介面呈現與互動 (The Interface)

數據轉化的終點，是將深藏在資料庫中的記錄，重新轉譯回「以人為本」的視覺介面。透過 HTML 與後端整合，SQL 記錄變成了直觀的儀表板。



Upload：



View Receipts：

A screenshot of the 'View Receipts' page. The browser's address bar shows 'smart-receipt-helper.onrender.com/receipts'. The page has a heading 'Receipt List' and a table with receipt data.

ID	Store	Invoice	Date	Amount
1	花蓮市農會自強生鮮超級市場	WH-37166026	2026-02-12	190
2	花蓮市農會自強生鮮超級市場	WH-05164124	2026-02-12	190

根據系統設計，使用者可體驗以下完整流程：

- **上傳與捕捉**：使用者透過 repo010.JPG 所示的簡單上傳按鈕，提交發票影像。
- **即時管理**：在「Receipt List」清單中，以表格形式檢視所有存檔，包含 ID、商店與金額。
- **跨平台互動**：整合 LINE Bot，讓 AI 自動將處理完成的摘要訊息推送到使用者的手機中。

核心洞察：優秀的介面設計能讓複雜的 AI 運算完全「透明化」。使用者不需要理解什麼是 OCR 或 JSON 定義，他們只需要「拍照」與「查看」，這正是運用科技將生活智慧化的最佳途徑。

7. 結語：

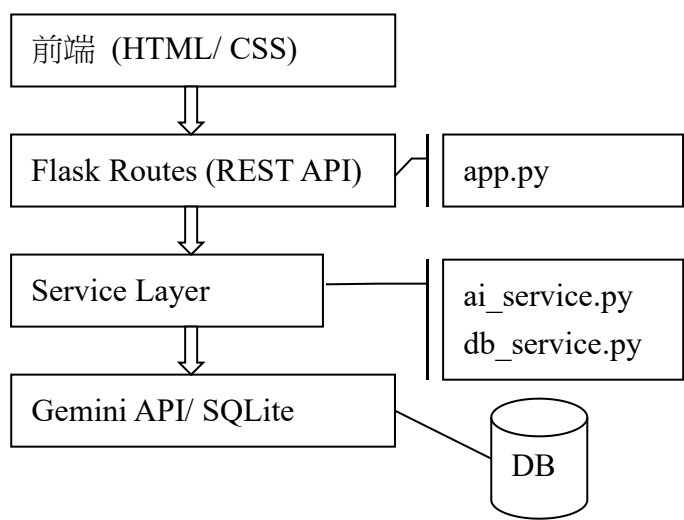
從一張物理發票到雲端資料庫，數據經歷了五層形態的蛻變：

1. **圖像 (Physical)**：現實世界的原始投影。
2. **文字 (Raw Text)**：透過 OCR 進行的初步數位化。
3. **JSON (Logic)**：經 AI 提煉後的結構化語義。
4. **資料庫 (Persistence)**：具備索引與唯一性的永久記憶。
5. **介面 (UI/UX)**：最終與人溝通的語言。

這套流程不僅是為了「自動記帳」，它更展示了 AI 如何將雜亂無章的現實轉化為具備數位智慧的資產。

完成基礎架構後，還可以衍生進階功能，例如「消費類別自動歸納」或「數據視覺化分析圖表」。

System Architecture



Repository

```
smart-receipt-helper/  
|  
|—— app.py  
|—— requirements.txt  
|—— README.md  
|  
|—— database/  
|   |—— receipts.db  
|  
|—— services/  
|   |—— ai_service.py  
|   |—— db_service.py  
|  
|—— static/  
|   |—— style.css  
|  
|—— templates/  
|   |—— index.html  
|   |—— upload.html  
|   |—— receipts.html
```